National University of Computer and Emerging Sciences (FAST–NUCES)

Breaking PsycoSupport

Offline deployment, loopback interception and Al-Kindi cryptanalysis of an affine-encrypted LLM support app

Information Security · Assignment #1 · Fall 2026

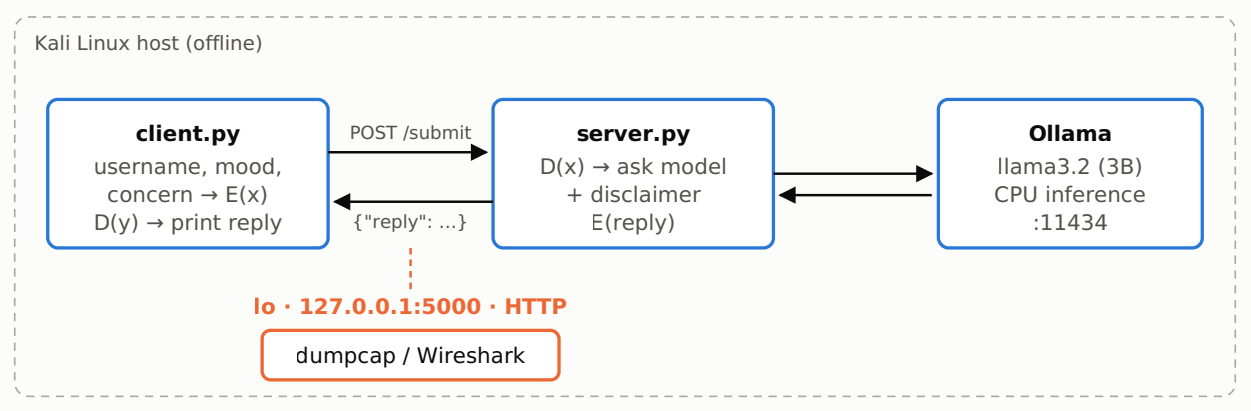
Muhammad Rafique · Roll No. 23i-0747

21 October 2026

Repository: github.com/Rafique610/psycosupport-cryptanalysis

Introduction

PsycoSupport is a local mental-health support tool: a patient enters a username, a mood and a concern; a server asks a local LLM (Ollama, `llama3.2`) for supportive advice; both directions are "protected" with an affine cipher. This report deploys the app fully offline (Task 1), intercepts its loopback traffic (Task 2), recovers the key and decrypts every session using only the intercepted ciphertext (Task 3), and analyses the security failures with proper alternatives (Task 4).



All results come from real runs on 21 October 2026 and are reproducible with the commands in the Appendix.

1. Offline Environment Deployment

1.1 What was installed

Component	Version	How
Kali GNU/Linux Rolling	kernel 7.1.5+kali-amd64	host OS
Ollama	0.40.7	official script `curl -fsSL https://ollama.com/install.sh | sh`, running as the `ollama` systemd service
Model `llama3.2` (3B)	ID `a80c4f17acd5`, 2.0 GB	`ollama pull llama3.2`
uv	0.13.4	Python environment from `pyproject.toml` + `uv.lock` (`ollama`, `requests`)
go-task	3.54.0	`Taskfile.yml` shortcuts (`task server`, `task client`, `task ollama-check`, `task offline-proof`)

The brief's install command (`curl -fsSL https://ollama.com | sh`) downloads the Ollama web page, not the installer; the real script lives at `/install.sh`.

1.2 Model choice

`llama3.2` (3B parameters) is the brief's first suggestion and fits the hardware comfortably, so `phi3` or `llama3.2:1b` were not needed. It is a small instruction-tuned model, which keeps replies fast on a laptop while still following the system prompt's style constraints.

1.3 Hardware

- CPU: 13th Gen Intel Core i5-13420H, 12 threads
- RAM: 15 GiB
- GPU: NVIDIA GeForce RTX 3050 6 GB Laptop GPU. Ollama detected it through Vulkan (open-source NVK driver), with about 5.4 GiB available for inference.

1.4 The PsycoSupport pipeline

The published repository contains only pre-built binaries whose server still calls the Groq cloud API, so `psycare_lab/` was written to the brief's specification:

- `server.py`: HTTP server bound to `127.0.0.1:5000`, endpoint `POST /submit`. It decrypts the patient's username, mood and concern, sends them to the local `llama3.2` model with a restrictive system prompt, and returns the encrypted reply.
- `client.py`: terminal client that encrypts the three fields, sends them, and decrypts the answer.
- `affine.py`: the Affine cipher both sides use.

1.5 Offline proof

On 21 October 2026 at 6:38 PM (UTC+05:00 · 13:38 UTC), networking was switched off with `nmcli networking off`:

1. `nmcli networking` reported `disabled`, and `curl https://ollama.com` failed with *Could not resolve host* (no internet).
2. `ollama run llama3.2 "Say hi in five words"` still answered: *Hello from the digital realm*.
3. `server.py` started and listened on `127.0.0.1:5000` (`ss -ltnp`).
4. A full client request (mood *stressed*, concern about exams) returned HTTP 200. The reply crossed the loopback as ciphertext and decrypted to a complete supportive answer.

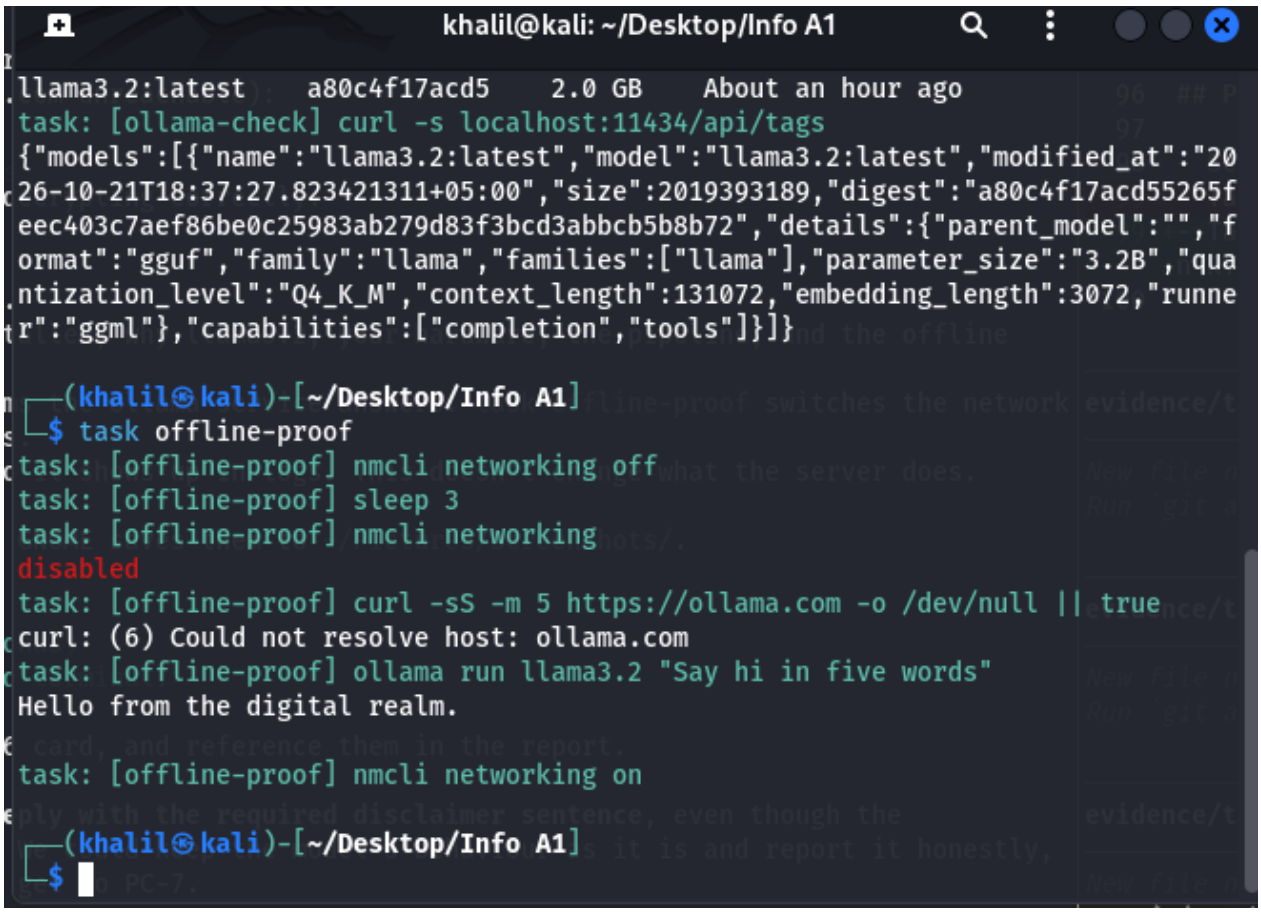
Networking was then switched back on.

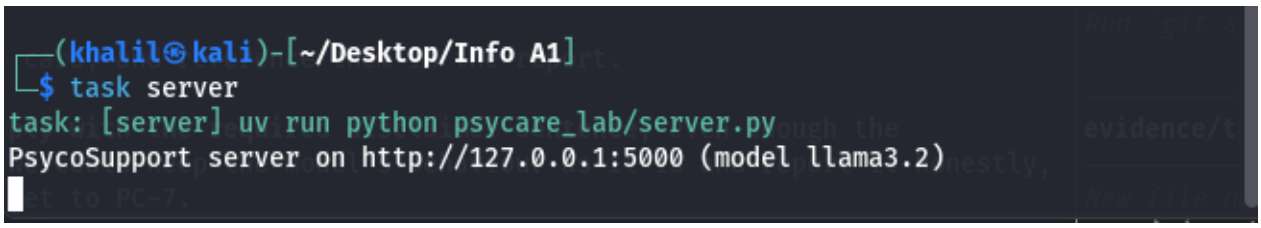
Evidence (text logs, `evidence/task1/`):

- `01-ollama-list.txt`: Ollama version and installed model
- `02-offline-ollama-run.txt`: networking disabled, no DNS, model answers
- `03-server-start.txt`: server startup
- `04-ss-port-5000.txt`: server bound to `127.0.0.1:5000`
- `05-offline-end-to-end.txt`: offline request, ciphertext on the wire, decrypted reply

Screenshots (taken 21 October 2026, 7:54 to 7:55 PM, UTC+05:00 · 14:54 to 14:55 UTC):




khalil@kali: ~/Desktop/Info A1
llama3.2:latest a80c4f17acd5 2.0 GB About an hour ago
task: [ollama-check] curl -s localhost:11434/api/tags
{"models":[{"name":"llama3.2:latest","model":"llama3.2:latest","modified_at":"2026-10-21T18:37:27.823421311+05:00","size":2019393189,"digest":"a80c4f17acd55265feec403c7aef86be0c25983ab279d83f3bcd3abbcb5b8b72","details":{"parent_model":"","format":"gguf","family":"llama","families":["llama"],"parameter_size":"3.2B","quantization_level":"Q4_K_M","context_length":131072,"embedding_length":3072,"runner":"ggml"},"capabilities":["completion","tools"]}]}
(khalil㉿kali)-[~/Desktop/Info A1]
$ task offline-proof
task: [offline-proof] nmcli networking off
task: [offline-proof] sleep 3
task: [offline-proof] nmcli networking
disabled
task: [offline-proof] curl -sS -m 5 https://ollama.com -o /dev/null || true
curl: (6) Could not resolve host: ollama.com
task: [offline-proof] ollama run llama3.2 "Say hi in five words"
Hello from the digital realm.
task: [offline-proof] nmcli networking on
(khalil㉿kali)-[~/Desktop/Info A1]
$


(khalil㉿kali)-[~/Desktop/Info A1]
$ task server
task: [server] uv run python psycare_lab/server.py
PsycoSupport server on http://127.0.0.1:5000 (model llama3.2)


(khalil㉿kali)-[~/Desktop/Info A1]
$ ss -ltnp | grep 5000
LISTEN 0 5 127.0.0.1:5000 0.0.0.0:* users:(("python",pid=50574,fd=3))

2. Local Network Traffic Interception

2.1 Setup

- Client and server run on the same Kali machine. The client connects to `http://muhammadrafique:5000` (the brief's "host file" name, mapped to `127.0.0.1` in `/etc/hosts`), so every byte crosses the loopback interface `lo` only.
- Capture tool: `dumpcap` (Wireshark's capture engine), run as a non-root member of the `wireshark` group:

```
bash dumpcap -i lo -f "tcp port 5000" -w evidence/task2/session_all.pcap # task capture
```

Capture filter `tcp port 5000`; in Wireshark the matching display filter is `tcp.port == 5000`. - The five test cases (`scripts/capture/test_cases.csv`) were fed to the real `client.py` by `scripts/capture/run_cases.py` (`task run-cases`), with a 5-second pause between sessions so each becomes a separate TCP stream. - Capture window: 21 October 2026, 8:33:25 to 8:34:36 PM (UTC+05:00 · 15:33:25 to 15:34:36 UTC). Result: 70 packets, 0 dropped, 14.1 KB (`evidence/task2/session_all.pcap`). The file is small enough to ship whole, so no `editcap` trimming was needed.

The model ran on the CPU for this capture: a first attempt on the RTX 3050 failed when the open-source nouveau driver timed out (`job timeout, channel 4 killed`), so `server.py` now requests CPU inference (`num_gpu: 0`, about 15 tokens/s).

2.2 Test cases (synthetic data only)

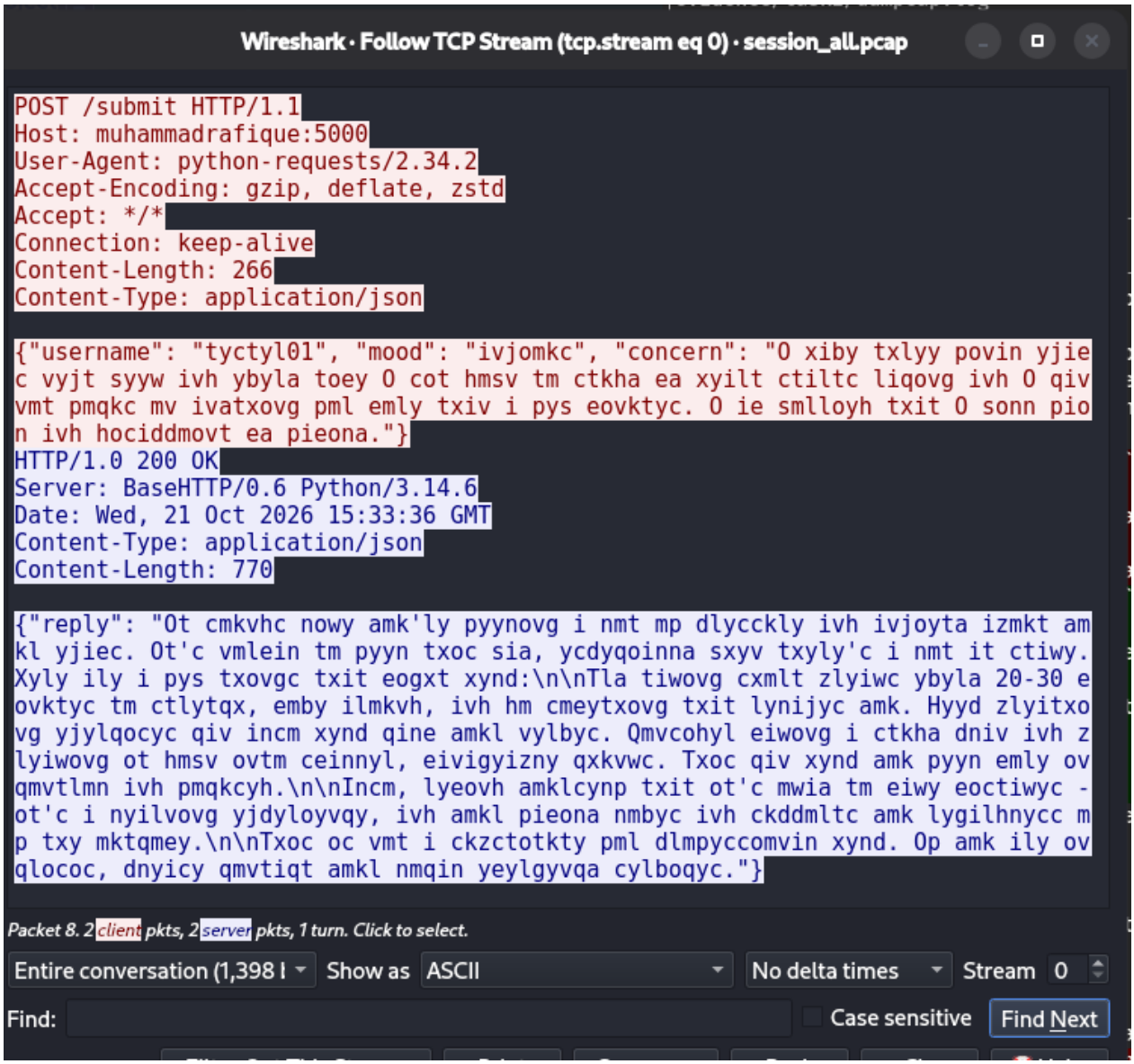
Stream	Username	Mood	Concern (summary)	Concern length
0	tester01	anxious	Three final exams next week, racing heart, fear of failing	208 chars
1	tester02	exhausted	A month of poor sleep, awake until 3 AM worrying	184 chars
2	tester03	lonely	New city for university, no friends yet	155 chars
3	tester04	overwhelmed	Manager keeps adding deadlines, working late	173 chars
4	tester05	unmotivated	Lost interest in painting and walks, low energy	159 chars

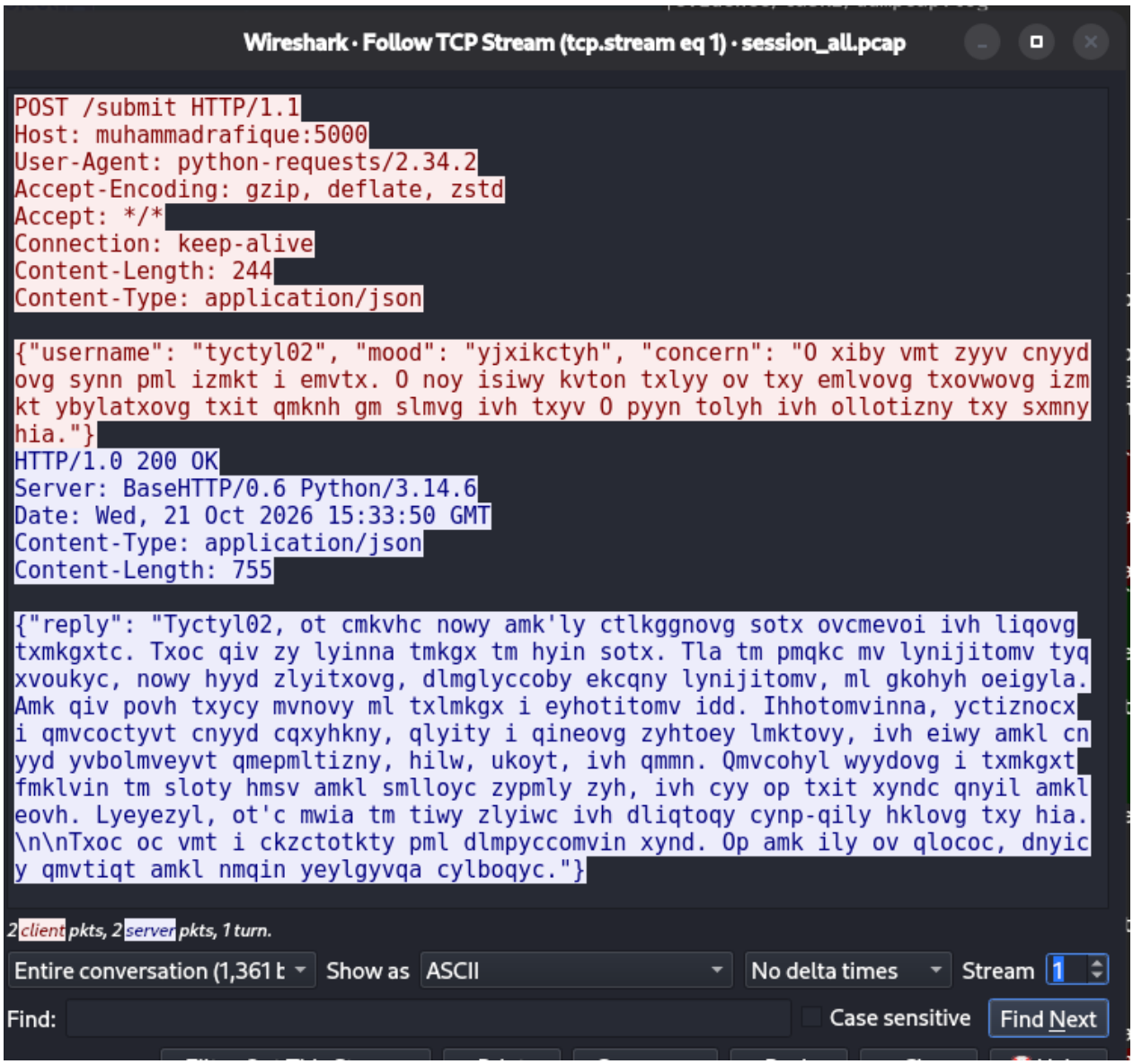
All names and concerns are invented. No real personal data was used (ethics rule 2).

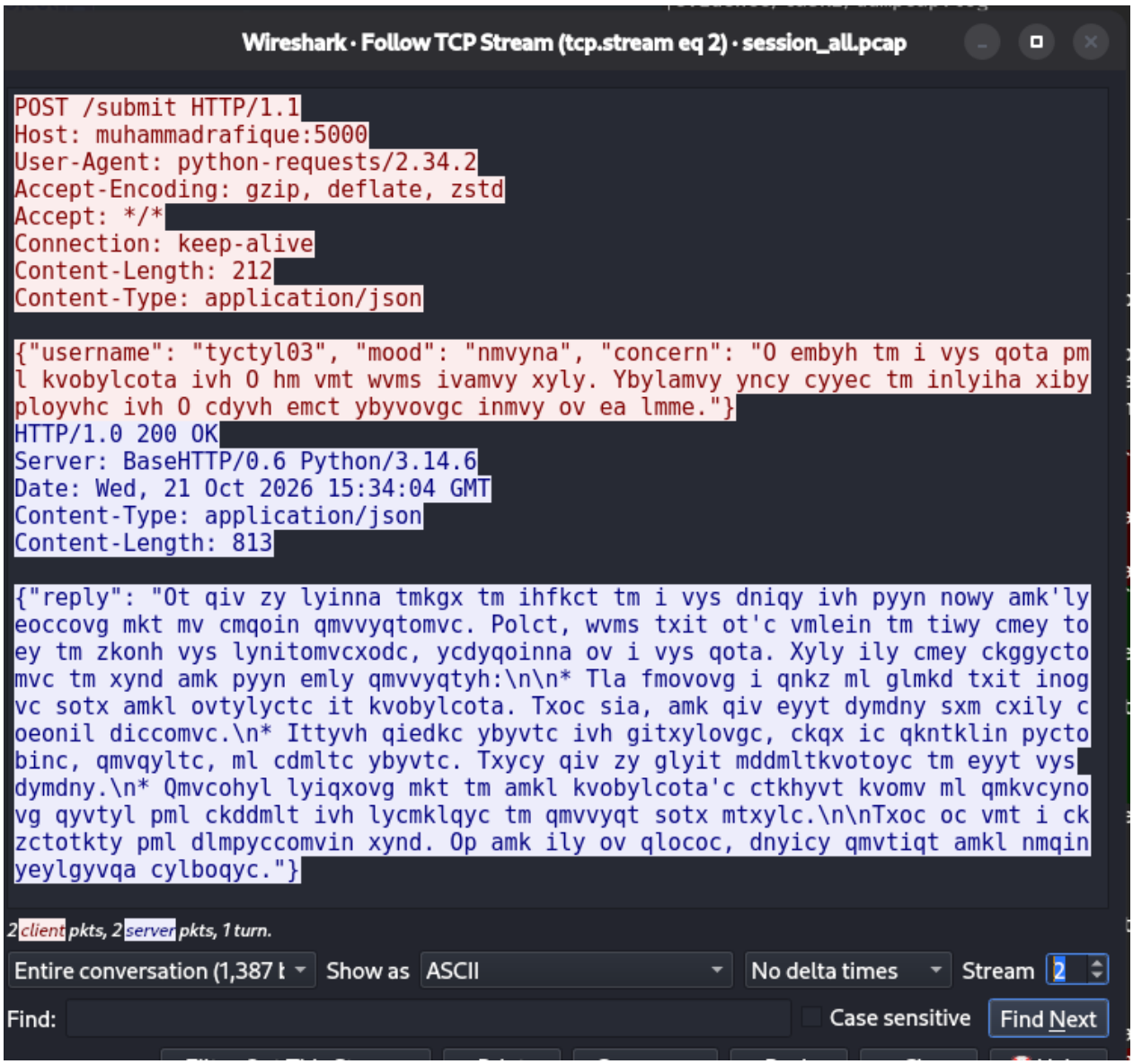
2.3 Captured streams

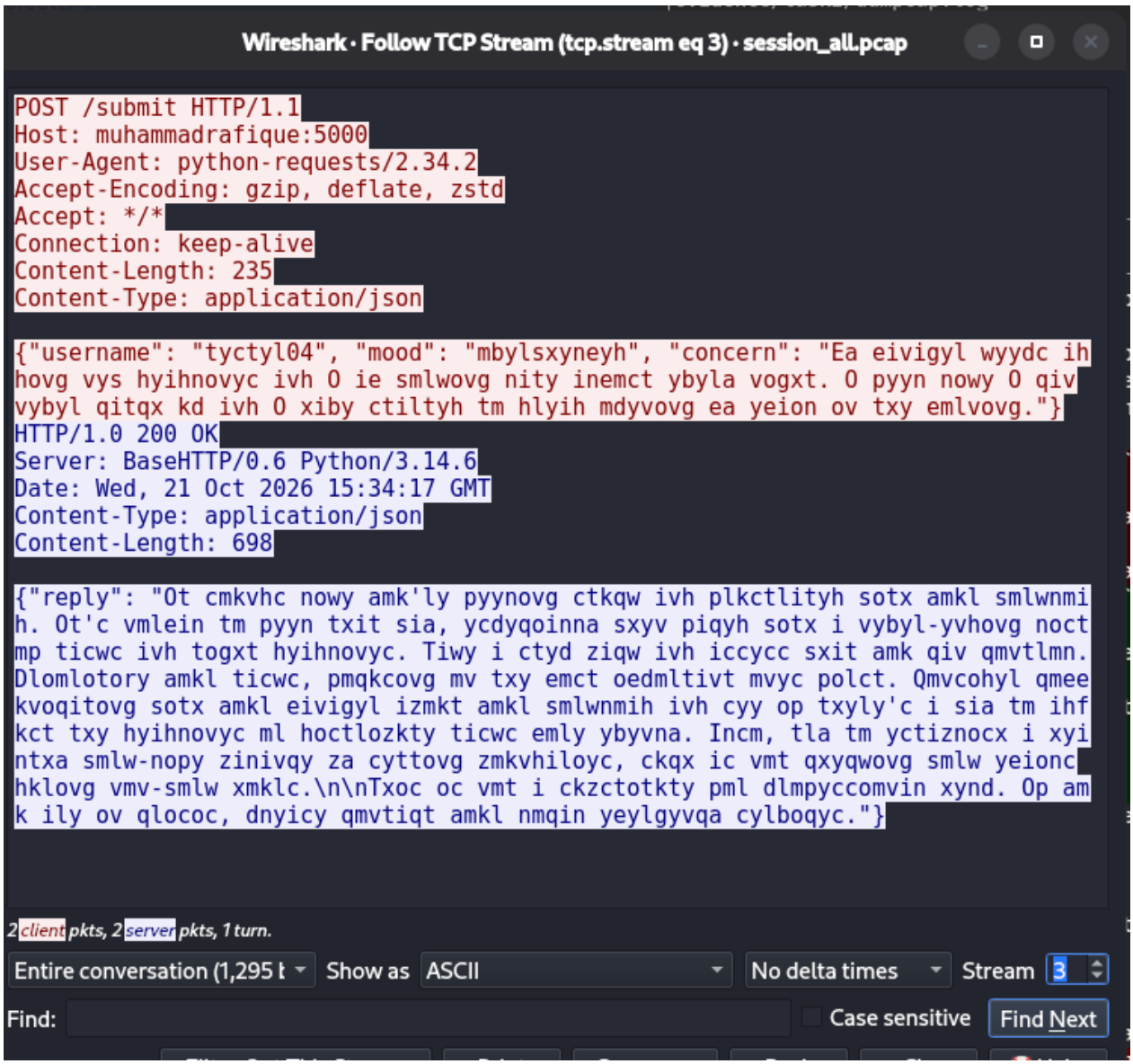
Stream	Client port	Client→Server bytes	Server→Client bytes
0	36372	483	915
1	50412	461	900
2	59644	429	958
3	53624	452	843
4	32770	438	803

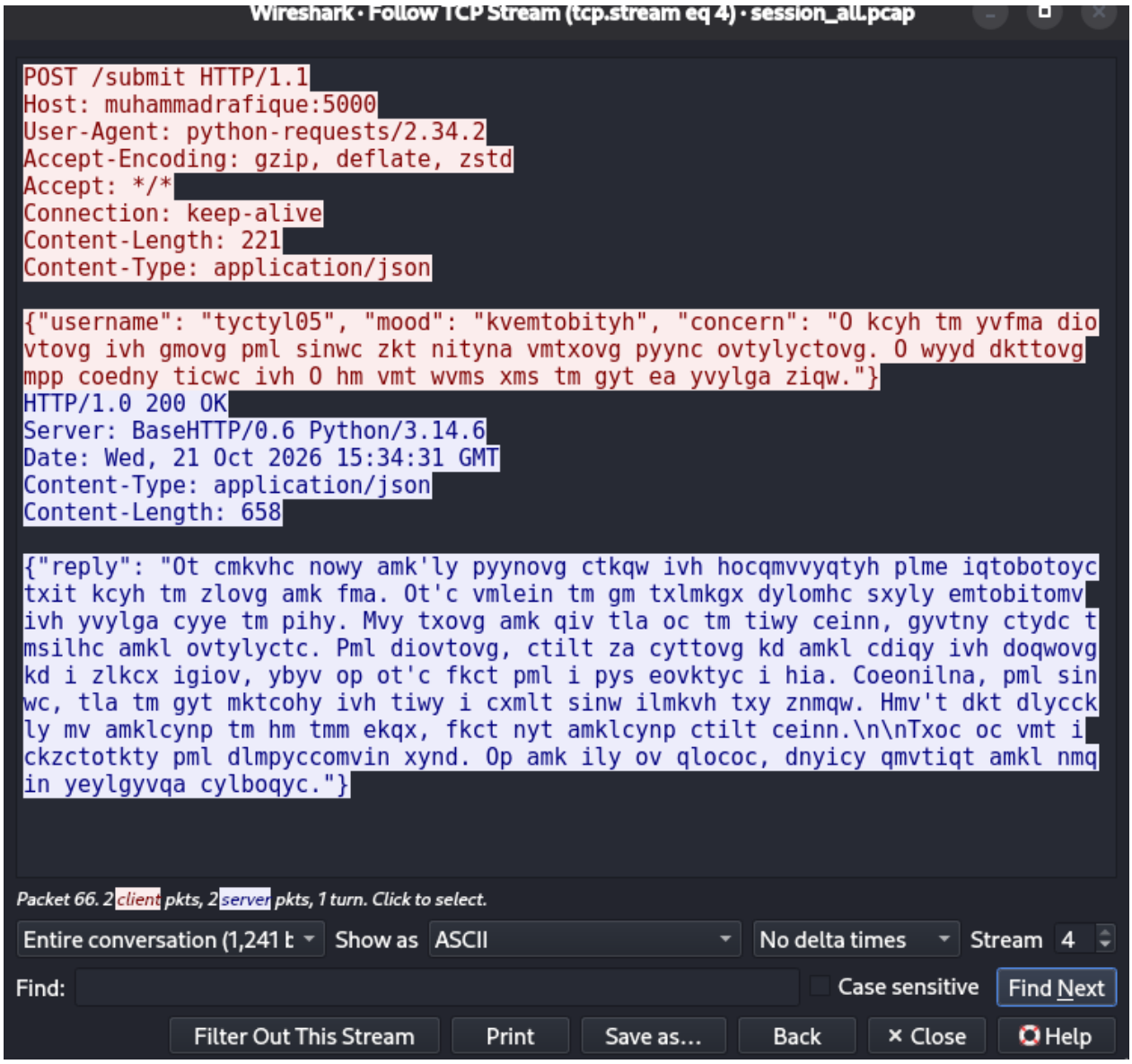
Raw Follow TCP Stream text (Wireshark *Save as…*, ASCII): `evidence/task2/stream_0.txt` to `stream_4.txt`.




Wireshark · Follow TCP Stream (tcp.stream eq 1) · session_all.pcap
POST /submit HTTP/1.1
Host: muhammadrafique:5000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate, zstd
Accept: */*
Connection: keep-alive
Content-Length: 244
Content-Type: application/json
{"username": "tyctyl02", "mood": "yjxikctyh", "concern": "O xiby vmt zyyv cnyyd ovg synn pml izmkt i emvtx. O noy isiwy kvton txlyy ov txy emlvovg txovwovg izmkt ybylatxovg txit qmknh gm slmvg ivh txyv O pyyn tolyh ivh ollotizny txy sxmny hia."}
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.6
Date: Wed, 21 Oct 2026 15:33:50 GMT
Content-Type: application/json
Content-Length: 755
{"reply": "Tyctyl02, ot cmkvhc nowy amk'ly ctlkggnovg sotx ovcmevoi ivh liqovg txmkgxtc. Txoc qiv zy lyinna tmkgx tm hyin sotx. Tla tm pmqkc mv lynijitomv tyq xvoukyc, nowy hyyd zlyitxovg, dlmglyccoby ekcqny lynijitomv, ml gkohyh oeigyla. Amk qiv povh txycy mvnovy ml txlmkgx i eyhotitomv idd. Ihhotomvinna, yctiznocx i qmvcoctyvt cnyyd cqxyhkny, qlyity i qineovg zyhtoey lmktovy, ivh eiwy amkl cnyyd yvbolmveyvt qmepmltizny, hilw, ukoyt, ivh qmmn. Qmvcohyl wyydovg i txmkgxt fmklvin tm sloty hmsv amkl smlloyc zypmly zyh, ivh cyy op txit xyndc qnyil amkl eovh. Lyeyezyl, ot'c mwia tm tiwy zlyiwc ivh dliqtoqy cynp-qily hklovg txy hia. \n\nTxoc oc vmt i ckzctotkty pml dlmpyccomvin xynd. Op amk ily ov qlococ, dnyic y qmvtiqt amkl nmqin yeylgyvqa cylboqyc."}
2 client pkts, 2 server pkts, 1 turn.
Entire conversation (1,361 t
Show as
ASCII
No delta times
Stream
1
Find:
Case sensitive
Find Next


Wireshark · Follow TCP Stream (tcp.stream eq 2) · session_all.pcap
POST /submit HTTP/1.1
Host: muhammadrafique:5000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate, zstd
Accept: */*
Connection: keep-alive
Content-Length: 212
Content-Type: application/json
{"username": "tyctyl03", "mood": "nmvyna", "concern": "O embyh tm i vys qota pm l kvobylcota ivh O hm vmt wvms ivamvy xyly. Ybylamvy yncy cyyec tm inlyiha xiby ployvhc ivh O cdyvh emct ybyvovgc inmvy ov ea lmme."}
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.6
Date: Wed, 21 Oct 2026 15:34:04 GMT
Content-Type: application/json
Content-Length: 813
{"reply": "Ot qiv zy lyinna tmkgx tm ihfkct tm i vys dniqy ivh pyyn nowy amk'ly eoccovg mkt mv cmqoin qmvvyqtomvc. Polct, wvms txit ot'c vmlein tm tiwy cmey to ey tm zkonh vys lynitomvcxodc, ycdyqoinna ov i vys qota. Xyly ily cmey ckggycto mvc tm xynd amk pyyn emly qmvvyqtyh:\n\n* Tla fmovovg i qnkz ml glmkd txit inog vc sotx amkl ovtylyctc it kvobylcota. Txoc sia, amk qiv eyyt dymdny sxm cxily c oeonil diccomvc.\n* Ittyvh qiedkc ybyvtc ivh gitxylovgc, ckqx ic qkntklin pycto binc, qmvqyltc, ml cdmltc ybyvtc. Txycy qiv zy glyit mddmltkvotoyc tm eyyt vys dymdny.\n* Qmvcohyl lyiqxovg mkt tm amkl kvobylcota'c ctkhyvt kvomv ml qmkvcyno vg qyvtyl pml ckddmlt ivh lycmklqyc tm qmvvyqt sotx mtxylc.\n\nTxoc oc vmt i ck zctotkty pml dlmpyccomvin xynd. Op amk ily ov qlococ, dnyicy qmvtiqt amkl nmqin yeylgyvqa cylboqyc."}
2 client pkts, 2 server pkts, 1 turn.
Entire conversation (1,387 t
Show as
ASCII
No delta times
Stream
2
Find:
Case sensitive
Find Next


Wireshark · Follow TCP Stream (tcp.stream eq 3) · session_all.pcap
POST /submit HTTP/1.1
Host: muhammadrafique:5000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate, zstd
Accept: */*
Connection: keep-alive
Content-Length: 235
Content-Type: application/json
{"username": "tyctyl04", "mood": "mbylsxyneyh", "concern": "Ea eivigyl wyydc ih hovg vys hyihnovyc ivh O ie smlwovg nity inemct ybyla vogxt. O pyyn nowy O qiv vybyl qitqx kd ivh O xiby ctiltyh tm hlyih mdyvovg ea yeion ov txy emlvovg."}
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.6
Date: Wed, 21 Oct 2026 15:34:17 GMT
Content-Type: application/json
Content-Length: 698
{"reply": "Ot cmkvhc nowy amk'ly pyynovg ctkqw ivh plkctlityh sotx amkl smlwnmih. Ot'c vmlein tm pyyn txit sia, ycdyqoinna sxyv piqyh sotx i vybyl-yvhovg noctmp ticwc ivh togxt hyihnovyc. Tiwy i ctyd ziqw ivh iccycc sxit amk qiv qmvtlmn. Dlomlotory amkl ticwc, pmqkcovg mv txy emct oedmltivt mvyc polct. Qmvcohyl qmeekvoqitovg sotx amkl eivigyl izmkt amkl smlwnmih ivh cyy op txyly'c i sia tm ihfkct txy hyihnovyc ml hoctlozkty ticwc emly ybyvna. Incm, tla tm yctiznocx i xyintxa smlw-nopy zinivqy za cyttovg zmkvhiloyc, ckqx ic vmt qxyqwovg smlw yeionchklovg vmv-smlw xmklc.\n\nTxoc oc vmt i ckzctotkty pml dlmpyccomvin xynd. Op amk ily ov qlococ, dnyicy qmvtiqt amkl nmqin yeylgyvqa cylboqyc."}
2 client pkts, 2 server pkts, 1 turn.
Entire conversation (1,295 b
Show as
ASCII
No delta times
Stream
3
Find:
Case sensitive
Find Next



2.4 Intercepted ciphertexts (`ciphertexts.csv`)

`scripts/capture/extract_ciphertexts.py` (`task extract`) reads the pcap with the Python standard library, reassembles each TCP stream per direction, and writes every encrypted JSON value to `ciphertexts.csv` exactly as it crossed the wire (JSON escapes such as `\n` kept). Streams appear in capture order, so stream 0 is case 1; the usernames' plaintext digits (`tyctyl01` … `tyctyl05`) confirm the mapping.

Result: **20 ciphertexts** (5 cases × 3 request fields + 1 reply), 963 characters client→server and 3,629 characters server→client. All 20 match the Wireshark Follow TCP Stream exports (`stream_<n>.txt`) character for character. The concern lengths equal the plaintext test cases (208, 184, 155, 173, 159), as expected from a cipher that substitutes letters one for one.

Case	Direction	Field	Length	Ciphertext (first 40 chars)
1	client_to_server	username	8	`tyctyl01`
1	client_to_server	mood	7	`ivjomkc`
1	client_to_server	concern	208	`O xiby txlyy povin yjiec vyjt syyw ivh y…`
1	server_to_client	reply	757	`Ot cmkvhc nowy amk'ly pyynovg i nmt mp d…`
2	client_to_server	username	8	`tyctyl02`
2	client_to_server	mood	9	`yjxikctyh`
2	client_to_server	concern	184	`O xiby vmt zyyv cnyydovg synn pml izmkt …`
2	server_to_client	reply	742	`Tyctyl02, ot cmkvhc nowy amk'ly ctlkggno…`
3	client_to_server	username	8	`tyctyl03`
3	client_to_server	mood	6	`nmvyna`
3	client_to_server	concern	155	`O embyh tm i vys qota pml kvobylcota ivh…`
3	server_to_client	reply	800	`Ot qiv zy lyinna tmkgx tm ihfkct tm i vy…`
4	client_to_server	username	8	`tyctyl04`
4	client_to_server	mood	11	`mbylsxyneyh`
4	client_to_server	concern	173	`Ea eivigyl wyydc ihhovg vys hyihnovyc iv…`
4	server_to_client	reply	685	`Ot cmkvhc nowy amk'ly pyynovg ctkqw ivh …`
5	client_to_server	username	8	`tyctyl05`
5	client_to_server	mood	11	`kvemtobityh`
5	client_to_server	concern	159	`O kcyh tm yvfma diovtovg ivh gmovg pml s…`
5	server_to_client	reply	645	`Ot cmkvhc nowy amk'ly pyynovg ctkqw ivh …`

2.5 Input analysis: what the wire reveals

Each session is one HTTP request and one HTTP response in plain text. Only the **JSON values** are encrypted:

Part	Visible on the wire?	Example from stream 0
HTTP method, path, version	Plaintext	`POST /submit HTTP/1.1`, `HTTP/1.0 200 OK`
Headers	Plaintext	`Host: muhammadrafique:5000`, `User-Agent: python-requests/2.34.2`, `Server: BaseHTTP/0.6 Python/3.14.6`, `Content-Length`
JSON keys	Plaintext	`username`, `mood`, `concern`, `reply`
JSON values	Affine-encrypted	`"username": "tyctyl01"`
Digits, spaces, punctuation inside values	Plaintext	the `01` in `tyctyl01`, apostrophes, commas, full stops
Letter case	Preserved	`Tyctyl02` at the start of a reply

Consequences that help the attack in Section 3:

- The keys tell the attacker exactly which field is which, so the short `mood` value (one English adjective) and the `username` can be studied separately.
- Word boundaries and punctuation survive, so word lengths and patterns (`0` = a one-letter word, `ot'c` = a contraction) are visible.
- Every reply ends with the same fixed disclaimer sentence, a known-plaintext crib.
- `Content-Length` leaks the exact message size, and the headers fingerprint the software (Python 3.14.6, python-requests 2.34.2).

2.6 Ethics

The capture was bound to the loopback interface of our own lab machine, filtered to the PsycoSupport port, and contains only our own synthetic sessions (ethics rules 1, 2 and 4).

Input Analysis

What the client sends. Every session is one HTTP `POST /submit` with three JSON fields, and one response with a single `reply` field. Only the letters inside the JSON *values* are encrypted; the request line, headers, JSON keys, digits, punctuation, spaces and letter case all travel in the clear (Section 2.5).

Field	Direction	Length in this lab	Encrypted letters	Role in the attack
`username`	client→server	8 chars	6 per session	Digits `01–05` stay plain, which maps streams to cases
`mood`	client→server	6–11 chars	one English adjective	Short; readable once a few letters are known
`concern`	client→server	155–208 chars	~120–160	Patient's private text, the main secret
`reply`	server→client	645–800 chars	509–636	Long, fluent English: the richest statistical signal

How input length and variety fed the attack. Frequency analysis needs enough letters for the English profile to show through. The five sessions gave 3,663 encrypted letters in total, and the volume was the difference between a clean break and a guess. Re-running the χ^2 key search on different slices of the capture shows this directly (lower χ^2 = more clearly English; all slices recover the correct key (17, 8)):

Slice	Encrypted letters	Winning key	Winner χ^2	Runner-up χ^2	Margin
All ciphertext	3,663	(17, 8)	163.4	5,296.7	32×
Client→Server only	775	(17, 8)	72.4	1,252.3	17×
Server→Client only	2,888	(17, 8)	173.8	4,019.2	23×
A single reply (session 1)	593	(17, 8)	—	—	—

Every slice breaks, including **one reply on its own** (≈500–640 letters). That is the key finding of the input analysis: the long LLM responses carry most of the signal. Each reply is several hundred letters of grammatical English with the same vocabulary patterns as the training corpus behind the standard frequency table, so even a single intercepted answer is enough. The short request fields alone would also have worked (775 letters, 17× margin), but they are the patient's private words, which is exactly what should have been hardest to read.

Why variety still mattered. Five different concerns kept the attack honest: the letter counts come from distinct messages, not one sentence repeated, so the recovered key is not an artefact of a single input. Variety also exposed the determinism of the cipher: because several replies open with the same words, those openings produced identical ciphertext across sessions (Section 3.3.3), a pattern that only becomes visible when the inputs differ but some output does not.

The attacker's shortcut. None of this statistical work was strictly necessary. Because the request fields keep their structure, an attacker reads word lengths and common words straight from the wire, and because every reply ends in the same fixed disclaimer, two guessed letter pairs from that known sentence pin the key outright (Section 3.2). Long, predictable input is what makes an affine cipher indefensible here.

3. Al-Kindi Classical Cryptanalysis

3.1 Frequency analysis

Everything in this section is derived from the intercepted ciphertext alone (`ciphertexts.csv`). It is reproduced by `task freq`, which runs `scripts/frequency/count.py`, `patterns.py` and `plot.py`.

3.1.1 The alphabet and modulus

Before counting, we establish which characters the cipher actually transforms. The capture shows:

- **Only letters change.** Digits pass through untouched: the usernames arrive as `tyctyl01` … `tyctyl05`, keeping `01` … `05`. Spaces, commas, full stops, apostrophes (`amk'ly`, `ot'c`), colons and the JSON newline escapes (`\n\n`) are all unchanged.
- **Case is preserved.** Sentence-initial capitals stay capitals (`Ot cmkvhc…`, `Tyctyl02, ot…`), and the standalone capital `O` appears where English has the one-letter word *I*.
- **Word lengths are preserved.** Every ciphertext has exactly the length of its plaintext field (for example the five concerns are 208, 184, 155, 173 and 159 characters, the same as the test inputs).

So the cipher maps the 26 letters onto themselves and leaves everything else alone: the alphabet is A–Z with **modulus m = 26**, applied the same way to upper and lower case. All counts below fold case and ignore non-letters.

3.1.2 Letter frequency tables

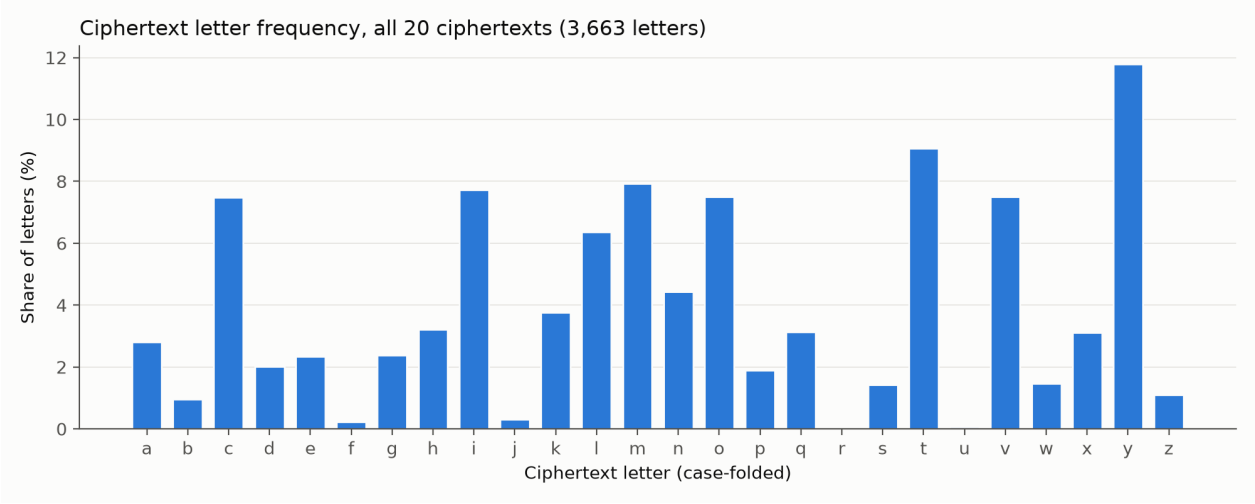
3,663 ciphertext letters were counted: 775 client→server (usernames, moods, concerns) and 2,888 server→client (the five LLM replies). Full tables: `report/figures/freq_table.csv`, `freq_table_client_to_server.csv`, `freq_table_server_to_client.csv` (columns symbol, count, percent, rank; the script asserts each table sums to 100 %).

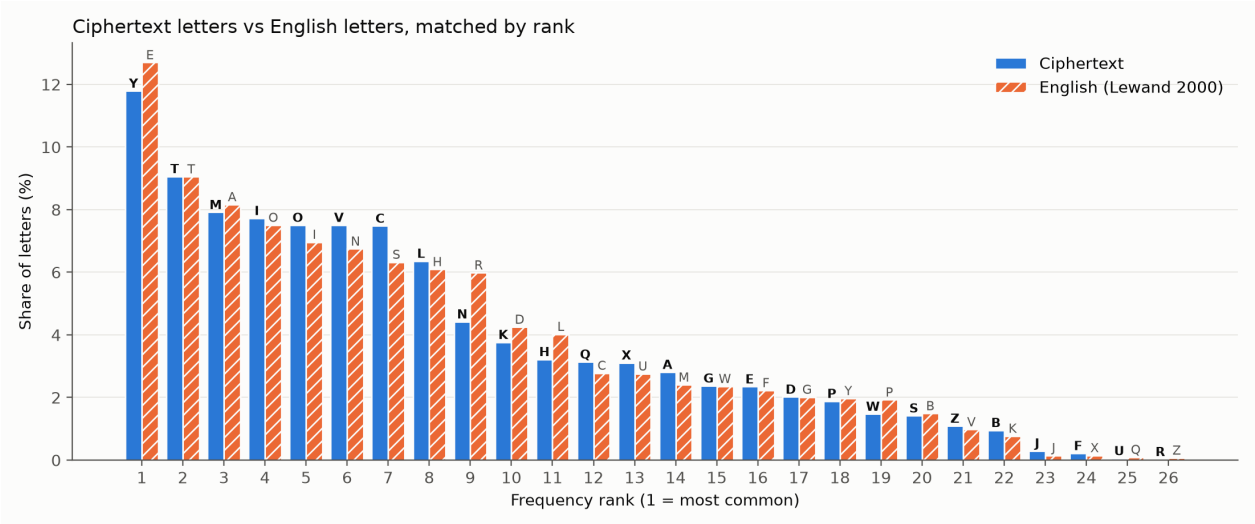
Rank	Cipher letter	Count	Cipher %	English letter (rank)	English %
1	y	432	11.79	e	12.70
2	t	332	9.06	t	9.06
3	m	290	7.92	a	8.17
4	i	283	7.73	o	7.51
5	o	275	7.51	i	6.97
6	v	275	7.51	n	6.75
7	c	274	7.48	s	6.33
8	l	233	6.36	h	6.09
9	n	162	4.42	r	5.99
10	k	138	3.77	d	4.25
11	h	118	3.22	l	4.03

English frequencies: R. E. Lewand, *Cryptological Mathematics*, Mathematical Association of America, 2000. The full 26-row pairing is in `report/figures/candidate_mapping.csv`.

Per direction, the top letter is the same:

Direction	Letters	Top 8 cipher letters (%)
client→server	775	y 13.5, v 9.9, t 9.2, o 8.5, i 8.0, m 6.7, l 5.2, c 4.5
server→client	2,888	y 11.3, t 9.0, c 8.3, m 8.2, i 7.7, o 7.2, v 6.9, l 6.7





3.1.3 Why this is a monoalphabetic substitution

1. **Same shape, different labels.** Sorted by frequency (Figure 3.2), the ciphertext profile tracks the English profile closely: one dominant letter near 12 %, a block of six or seven letters between 6 % and 9 %, then a long tail down to almost zero. Only the letter names differ. A polyalphabetic cipher (Vigenère, for example) would flatten this curve.
2. **Index of coincidence.** The probability that two randomly chosen letters are equal is unchanged by any one-to-one letter substitution:

Text	IC
All ciphertext	0.0647
client→server	0.0681
server→client	0.0646
English (from the Lewand table)	0.0655
Uniformly random letters	0.0385

The ciphertext sits on the English value, far from random. Each plaintext letter therefore always becomes the same ciphertext letter: exactly the behaviour of an affine cipher E(x) = (a·x + b) mod 26. 3. **Repetition survives.** Identical plaintext produces identical ciphertext (next section), which a cipher with a changing key would not do.

3.1.4 Structural patterns (the LLM gives itself away)

From `report/figures/patterns.md`:

Pattern	Ciphertext	Count	Most likely plaintext
Most common word	`ivh`	33	*and*
Most common trigram	`ovg` (mostly word-final)	48	*ing*
Common bigram / trigram	`tx`, `txy`	59, 15	*th*, *the*
One-letter words	`i`, `o` (capital `O`)	28, 18	*a*, *I*
Frequent 3- and 4-letter words	`amk`, `amkl`, `pml`	14, 19, 13	*you*, *your*, *for*
Letter after an apostrophe	`c` in `ot'c`, `t` in `hmv't`	11, 1	*s* (*it's*), *t* (*don't*)

The strongest structural clue is a **118-character block shared by all five replies**, always at the very end:

```
Txoc oc vmt i ckzctotkty pml dlmpyccomvin xynd. Op amk ily ov qlococ, dnyicy qmvtiqt
amkl nmqin yeylgyvqa cylboqyc.
```

Fixed boilerplate repeated across sessions is exactly the weakness the brief describes: LLM output ends in predictable disclaimers. Its word shapes (4-2-3-1-10-3-12-4, then a conditional *If you are in …*) give a long known-plaintext crib to confirm any key guess.

3.1.5 Candidate mapping for key recovery

Rank matching alone (Figure 3.2) is only a starting point: ranks 3 to 7 are within about 0.5 % of each other, so their order is noise. Combining frequency with structure gives the pairs we trust most:

Cipher	→ Plain	Evidence
y	e	Rank 1 in every direction (11.8 %), and the last letter of `txy` (*the*)
t	t	Rank 2 (9.06 % vs English 9.06 %), first letter of `txy` and `txit` (*that*)
x	h	Second letter of `txy`, `tx` the third most common bigram
i	a	One-letter word `i`, first letter of `ivh` (*and*)
v	n	Middle of `ivh`, and of `ovg` (*ing*)
h	d	Last letter of `ivh`
o	i	Capital `O` alone is the pronoun *I*; first letter of `ovg`
g	g	Last letter of `ovg`
m	o	Middle of `amk` (*you*) and `pml` (*for*), `tm` (*to*) 30 times

The two highest-confidence pairs, **y → e** (most frequent letter) and **t → t** (second most frequent, confirmed by *the*), are taken forward to Section 3.2, where they give two linear equations in a and b. Every other pair above then serves as an independent check on the recovered key.

3.2 Recovering the affine key (a, b)

The key is derived from the ciphertext statistics of Section 3.1 alone. The complete working below is printed by `task recover` (`scripts/cipher/recover_key.py`), which reads the two top-ranked letters from `report/figures/candidate_mapping.csv` and stores its output in `report/figures/key_recovery.txt`. Every run gives the same result.

3.2.1 Model

Letters are numbered a = 0, b = 1, …, z = 25 (Section 3.1.1 established m = 26). The affine cipher is

$E(x) = (a \cdot x + b) \bmod 26$, $D(y) = a^{-1} \cdot (y - b) \bmod 26$,

which is only decryptable when gcd(a, 26) = 1, so $a \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ (12 choices) and $b \in \{0, \ldots, 25\}$: $12 \times 26 = 312$ possible keys.

3.2.2 Two equations from the frequency analysis

The two most frequent ciphertext letters (Table 3.1) are assumed to be the two most frequent English letters:

Assumption	Cipher letter	c	Plain letter	p
Rank 1	y	24	e	4
Rank 2	t	19	t	19

Since E(p) = c:

(1) $4a + b \equiv 24 \pmod{26}$ (2) $19a + b \equiv 19 \pmod{26}$

Subtracting (2) from (1) removes b:

$(4 - 19)\cdot a \equiv 24 - 19 \Rightarrow -15a \equiv 5 \Rightarrow$ **$11a \equiv 5 \pmod{26}$** (since $-15 \equiv 11$)

3.2.3 Inverse of 11 modulo 26 (Extended Euclidean Algorithm)

Step	Division
1	$26 = 2 \times 11 + 4$
2	$11 = 2 \times 4 + 3$
3	$4 = 1 \times 3 + 1$
4	$3 = 3 \times 1 + 0$

The last non-zero remainder is 1, so gcd(26, 11) = 1 and 11 is invertible. Back-substitution:

$1 = 4 - 1\cdot 3 = 4 - (11 - 2\cdot 4) = 3\cdot 4 - 11 = 3\cdot(26 - 2\cdot 11) - 11 = 3\cdot 26 - 7\cdot 11$

So $-7\cdot 11 \equiv 1 \pmod{26}$, giving **$11^{-1} \equiv -7 \equiv 19 \pmod{26}$**. Check: $11 \times 19 = 209 = 8 \times 26 + 1 \equiv 1$. ✓

3.2.4 Solving for a and b

$a \equiv 19 \times 5 = 95 = 3 \times 26 + 17 \Rightarrow$ **$a = 17$**

$b \equiv 24 - 17 \times 4 = 24 - 68 = -44 \equiv -44 + 52 \Rightarrow$ **$b = 8$**

Check both equations: $E(4) = 17\cdot 4 + 8 = 76 = 2\cdot 26 + 24 \rightarrow$ `y` ✓; $E(19) = 17\cdot 19 + 8 = 331 = 12\cdot 26 + 19 \rightarrow$ `t` ✓. (The letter *t* is a fixed point of this key, which is why `t` appears unchanged in the ciphertext.)

3.2.5 Validity of a and the decryption key a^{-1}

gcd(17, 26) = 1, so a = 17 is a valid multiplier. Its inverse:

Step	Division
1	$26 = 1 \times 17 + 9$
2	$17 = 1 \times 9 + 8$
3	$9 = 1 \times 8 + 1$
4	$8 = 8 \times 1 + 0$

Back-substitution:

$1 = 9 - 8 = 9 - (17 - 9) = 2\cdot 9 - 17 = 2\cdot(26 - 17) - 17 = 2\cdot 26 - 3\cdot 17$

So **$a^{-1} \equiv -3 \equiv 23 \pmod{26}$**. Proof: $17 \times 23 = 391 = 15 \times 26 + 1 \equiv 1 \pmod{26}$. ✓

The decryption function is therefore

$D(y) = 23\cdot(y - 8) \bmod 26.$

3.2.6 Validation

Independent letter checks. Section 3.1.5 proposed seven more pairs from word structure, none of them used to build the equations. The recovered key confirms all seven:

Cipher	y	D(y) = 23(y − 8) mod 26	Plain	Structural guess
x	23	23·15 = 345 ≡ 7	h	*h* in `txy` = the ✓
i	8	23·0 = 0	a	*a* ✓
v	21	23·13 = 299 ≡ 13	n	*n* in `ivh` = and ✓
h	7	23·(−1) = −23 ≡ 3	d	*d* in `ivh` = and ✓
o	14	23·6 = 138 ≡ 8	i	*I* / *ing* ✓
g	6	23·(−2) = −46 ≡ 6	g	*g* in `ovg` = ing ✓
m	12	23·4 = 92 ≡ 14	o	*o* in `tm` = to ✓

Decrypted samples.

- Reply 1, `Ot cmkvhc nowy amk'ly pyynovg i nmt mp dlycckly…` → *It sounds like you're feeling a lot of pressure and anxiety about your exams. It's normal to feel this way...*
- The block shared by all five replies (Section 3.1.4) → *This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.*

Chi-squared cross-check. As an independent check only, all 312 valid keys were tried on the full ciphertext, and each decryption was scored against English letter frequencies with $\chi^2 = \Sigma\ (\text{observed}_i - \text{expected}_i)^2 / \text{expected}_i$ (lower means more English-like):

Rank	a	b	χ^2
1	**17**	**8**	**163.4** (recovered key)
2	15	24	5,296.7
3	7	24	5,777.4
4	1	6	6,364.8
5	23	0	6,920.0
6	15	11	7,228.1

The recovered key is the clear winner, about 32 times better than the runner-up. The script asserts that the frequency-derived key and the χ^2 winner agree.

Confirmation against the source. Only after the derivation above was the key in `psycare_lab/affine.py` inspected: `A = 17`, `B = 8`, `A_INV = 23`, identical to the recovered values.

Recovered key: (a, b) = (17, 8), with decryption multiplier $a^{-1} = 23$.

3.2.7 Why this was easy

- **Two letters were enough.** The affine cipher has only 312 keys and is fully determined by two plaintext–ciphertext letter pairs. Frequency analysis supplied them from the first two ranks.
- **The LLM's output is predictable English.** Long, grammatical replies push the letter distribution towards textbook English (IC 0.0647 vs 0.0655), so the top-ranked letters land exactly where expected (`t` matches English *t* to two decimal places).
- **Boilerplate is free known plaintext.** The fixed 118-character disclaimer at the end of every reply turns a ciphertext-only attack into a known-plaintext one: an attacker who guesses the standard *This is not a substitute for professional help* can read off a dozen letter pairs and confirm the key without any statistics.
- **Structure leaks through.** Spaces, punctuation, case and JSON keys survive encryption, exposing word lengths and common words (*and*, *the*, *-ing*).

These failures are analysed further in Task 4.

3.3 Decrypting all five sessions

With the recovered key (a, b) = (17, 8), every ciphertext is decrypted with D(y) = 23·(y − 8) mod 26. `task decrypt` (`scripts/cipher/decrypt_all.py`) processes all 20 rows of `ciphertexts.csv` and writes `report/figures/decrypted.csv` (case_id, direction, field, ciphertext, plaintext). JSON escapes are undone before decrypting, so the `n` of an escaped newline is never treated as a cipher letter.

3.3.1 Verification against ground truth

Two independent references exist for every row:

- **Client→Server:** the original inputs in `scripts/capture/test_cases.csv`. Each decrypted field must be identical.
- **Server→Client:** the reply the real PsycoSupport client printed during the capture (`evidence/task2/client_runs.txt`). Each decrypted reply must appear there character for character.

```
$ task decrypt
case 1: client_to_server PASS (vs test_cases.csv) | server_to_client PASS (vs client
output in client_runs.txt)
case 2: client_to_server PASS (vs test_cases.csv) | server_to_client PASS (vs client
output in client_runs.txt)
case 3: client_to_server PASS (vs test_cases.csv) | server_to_client PASS (vs client
output in client_runs.txt)
case 4: client_to_server PASS (vs test_cases.csv) | server_to_client PASS (vs client
output in client_runs.txt)
case 5: client_to_server PASS (vs test_cases.csv) | server_to_client PASS (vs client
output in client_runs.txt)
identical disclaimer ciphertext in 5/5 replies
wrote 20 rows to report/figures/decrypted.csv
```

Result: 20/20 ciphertexts decrypted correctly: 5/5 sessions in both directions.

3.3.2 Session by session

Session 1: exam stress (stream 0) · PASS

Direction	Field	Ciphertext	Plaintext
Client→Server	username	`tyctyl01`	tester01
Client→Server	mood	`ivjomkc`	anxious
Client→Server	concern	`O xiby txlyy povin yjiec vyjt syyw ivh ybyla toey O cot hmsv…`	I have three final exams next week and every time I sit down to study my heart starts racing and I cannot focus on anything for more than a few minutes. I am worried that I will fail and disappoint my family.
Server→Client	reply	`Ot cmkvhc nowy amk'ly pyynovg i nmt mp dlycckly ivh ivjoyta …`	(full text below)

Decrypted reply:

> It sounds like you're feeling a lot of pressure and anxiety about your exams. It's normal to feel this way, especially when there's a lot at stake. Here are a few things that might help:
>
> Try taking short breaks every 20-30 minutes to stretch, move around, and do something that relaxes you. Deep breathing exercises can also help calm your nerves. Consider making a study plan and breaking it down into smaller, manageable chunks. This can help you feel more in control and focused.
>
> Also, remind yourself that it's okay to make mistakes - it's a learning experience, and your family loves and supports you regardless of the outcome.
>
> This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.

Session 2: poor sleep (stream 1) · PASS

Direction	Field	Ciphertext	Plaintext
Client→Server	username	`tyctyl02`	tester02
Client→Server	mood	`yjxikctyh`	exhausted
Client→Server	concern	`O xiby vmt zyyv cnyydovg synn pml izmkt i emvtx. O noy isiwy…`	I have not been sleeping well for about a month. I lie awake until three in the morning thinking about everything that could go wrong and then I feel tired and irritable the whole day.
Server→Client	reply	`Tyctyl02, ot cmkvhc nowy amk'ly ctlkggnovg sotx ovcmevoi ivh…`	(full text below)

Decrypted reply:

> Tester02, it sounds like you're struggling with insomnia and racing thoughts. This can be really tough to deal with. Try to focus on relaxation techniques, like deep breathing, progressive muscle relaxation, or guided imagery. You can find these online or through a meditation app. Additionally, establish a consistent sleep schedule, create a calming bedtime routine, and make your sleep environment comfortable, dark, quiet, and cool. Consider keeping a thought journal to write down your worries before bed, and see if that helps clear your mind. Remember, it's okay to take breaks and practice self-care during the day.
>
> This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.

Session 3: loneliness (stream 2) · PASS

Direction	Field	Ciphertext	Plaintext
Client→Server	username	`tyctyl03`	tester03
Client→Server	mood	`nmvyna`	lonely
Client→Server	concern	`O embyh tm i vys qota pml kvobylcota ivh O hm vmt wvms ivamv…`	I moved to a new city for university and I do not know anyone here. Everyone else seems to already have friends and I spend most evenings alone in my room.
Server→Client	reply	`Ot qiv zy lyinna tmkgx tm ihfkct tm i vys dniqy ivh pyyn now…`	(full text below)

Decrypted reply:

> It can be really tough to adjust to a new place and feel like you're missing out on social connections. First, know that it's normal to take some time to build new relationships, especially in a new city. Here are some suggestions to help you feel more connected:
>
> - Try joining a club or group that aligns with your interests at university. This way, you can meet people who share similar passions.
> - Attend campus events and gatherings, such as cultural festivals, concerts, or sports events. These can be great opportunities to meet new people.
> - Consider reaching out to your university's student union or counseling center for support and resources to connect with others.
>
> This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.

Session 4: work pressure (stream 3) · PASS

Direction	Field	Ciphertext	Plaintext
Client→Server	username	`tyctyl04`	tester04
Client→Server	mood	`mbylsxyneyh`	overwhelmed
Client→Server	concern	`Ea eivigyl wyydc ihhovg vys hyihnovyc ivh O ie smlwovg nity …`	My manager keeps adding new deadlines and I am working late almost every night. I feel like I can never catch up and I have started to dread opening my email in the morning.
Server→Client	reply	`Ot cmkvhc nowy amk'ly pyynovg ctkqw ivh plkctlityh sotx amkl…`	(full text below)

Decrypted reply:

> It sounds like you're feeling stuck and frustrated with your workload. It's normal to feel that way, especially when faced with a never-ending list of tasks and tight deadlines. Take a step back and assess what you can control. Prioritize your tasks, focusing on the most important ones first. Consider communicating with your manager about your workload and see if there's a way to adjust the deadlines or distribute tasks more evenly. Also, try to establish a healthy work-life balance by setting boundaries, such as not checking work emails during non-work hours.
>
> This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.

Session 5: low motivation (stream 4) · PASS

Direction	Field	Ciphertext	Plaintext
Client→Server	username	`tyctyl05`	tester05
Client→Server	mood	`kvemtobityh`	unmotivated
Client→Server	concern	`O kcyh tm yvfma diovtovg ivh gmovg pml sinwc zkt nityna vmtx…`	I used to enjoy painting and going for walks but lately nothing feels interesting. I keep putting off simple tasks and I do not know how to get my energy back.
Server→Client	reply	`Ot cmkvhc nowy amk'ly pyynovg ctkqw ivh hocqmvvyqtyh plme iq…`	(full text below)

Decrypted reply:

> It sounds like you're feeling stuck and disconnected from activities that used to bring you joy. It's normal to go through periods where motivation and energy seem to fade. One thing you can try is to take small, gentle steps towards your interests. For painting, start by setting up your space and picking up a brush again, even if it's just for a few minutes a day. Similarly, for walks, try to get outside and take a short walk around the block. Don't put pressure on yourself to do too much, just let yourself start small.
>
> This is not a substitute for professional help. If you are in crisis, please contact your local emergency services.

3.3.3 Identical plaintext, identical ciphertext

The cipher has no initialisation vector, nonce or per-message randomness: the same plaintext always produces the same ciphertext, in every session.

Plaintext	Sessions	Ciphertext (the same every time)
It sounds like you're feeling	1, 4, 5	`Ot cmkvhc nowy amk'ly pyynovg`
It's normal to	1, 4, 5	`Ot'c vmlein tm`
This is not a substitute for professional help.	1, 2, 3, 4, 5	`Txoc oc vmt i ckzctotkty pml dlmpyccomvin xynd.`

An eavesdropper does not even need the key to learn something: spotting the same ciphertext opener in several sessions already reveals that the replies start the same way, and the fixed 118-character disclaimer marks exactly where each reply ends. Combined with the key, all 20 intercepted messages, including each patient's username, mood and private concern, are fully readable.

4. Security Engineering Analysis

PsycoSupport promises "end-to-end encryption" for patients' mental-health concerns. In this lab an unprivileged observer on the same machine recovered the key in under a second and read every message. This section explains why, and how local inter-process communication (IPC) should be protected instead.

4.1 Why LLM output breaks classical ciphers

The affine cipher is a **monoalphabetic substitution**: each plaintext letter always maps to the same ciphertext letter, and nothing else changes. LLM-generated text is close to the worst possible input for such a cipher, because it is long, fluent and formulaic.

1. Text that looks like the textbook. Five replies gave 2,888 cipher letters whose index of coincidence (0.0646) matches English (0.0655) almost exactly (Section 3.1.3). The most frequent cipher letter `y` sat at 11.8 % against English *e* at 12.7 %, and cipher `t` at 9.06 % against English *t* at 9.06 %. The first two ranks were enough to solve for the key (Section 3.2).

2. Boilerplate is known plaintext. Every reply ended with the same 118-character sentence (Section 3.1.4), which encrypted to the same ciphertext in all five sessions (Section 3.3.3):

Plaintext (repeated by the model or the app)	Sessions	Ciphertext, identical every time
This is not a substitute for professional help.	1–5	`Txoc oc vmt i ckzctotkty pml dlmpyccomvin xynd.`
It sounds like you're feeling	1, 4, 5	`Ot cmkvhc nowy amk'ly pyynovg`
It's normal to	1, 4, 5	`Ot'c vmlein tm`

An attacker who guesses the standard disclaimer gets dozens of plaintext–ciphertext letter pairs for free: a **known-plaintext attack**. Only **two** independent pairs are needed, because the key (a, b) is the solution of two linear congruences.

3. Deterministic encryption. There is no IV, nonce or per-message randomness. Equal plaintext always gives equal ciphertext, so repeated openers, list markers (`*`), and line breaks (`\n\n`) form visible patterns even before the key is known.

4. Structure survives. Spaces, punctuation, digits and letter case pass through unchanged. Word lengths and common words (`ivh` = *and*, 33 times; `ovg` = *-ing*, 48 times; `txy` = *the*) were readable straight from the capture.

5. A tiny keyspace. With m = 26 there are only 12 × 26 = **312** keys. Trying every one of them on the full 4,574-character capture took **0.14 s** on the lab laptop, and a χ^2 test picked the right key with a score 32 times better than the runner-up (Section 3.2.6). Frequency analysis is not even necessary: brute force alone breaks this cipher instantly.

4.2 Other failures observed

Failure	Evidence from this lab	Consequence
Metadata in the clear	HTTP request line, `Host: muhammadrafique:5000`, `User-Agent: python-requests/2.34.2`, `Server: BaseHTTP/0.6 Python/3.14.6`, JSON keys `username/mood/concern/reply`, exact `Content-Length` (Section 2.5)	An observer learns who talks to whom, which software runs, which field is which, and how long each message is, without decrypting anything
No integrity or authentication	Any ciphertext can be altered and re-sent; the server accepts any `POST /submit` from any local process; nothing proves the reply came from the real server	With the recovered key an attacker can forge requests or replace a reply with harmful "advice" undetected. Even without the key, bytes can be altered or replayed
Loopback is not private	`dumpcap` captured all 70 packets as an ordinary user in the `wireshark` group, no root needed (Section 2.1)	Any account or process with capture rights on the machine can read every session
Homemade cryptography	A classical cipher hard-coded in `affine.py`, shared between client and server	Violates Kerckhoffs's principle: security rests on the scheme staying secret, and the "key" is in the source code. Once seen, every past and future message is exposed, with no key rotation or forward secrecy
Sensitive data at stake	Usernames, moods and concerns such as panic before exams or loneliness (Section 3.3.2)	Mental-health information is special-category personal data. A leak harms patients directly

4.3 Protecting local IPC: alternatives

4.3.1 Alternative 1: UNIX domain socket with file permissions

How it works. Client and server talk through a socket file such as `/run/user/1000/psycare.sock` instead of TCP port 5000. The kernel copies data directly between the two processes. Nothing is ever sent over a network interface, so a capture on `lo` sees nothing. Access is controlled by ordinary file permissions: with mode `0600`, only the owning user can connect. Over the open connection, the server can ask the kernel who the peer is with `SO_PEERCRED`, which returns the client's PID, UID and GID, and refuse anyone unexpected (`man 7 unix`).

What it fixes. Sniffing on `lo` (the attack in Task 2), connections from other users' processes, and spoofed clients. It needs no certificates or keys.

Setup sketch (Python standard library):

```
import os, socket, struct

PATH = f"/run/user/{os.getuid()}/psycare.sock"   # per-user runtime directory, mode
0700
srv = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
os.umask(0o177)                                  # socket file is created as 0600
srv.bind(PATH)
srv.listen()
conn, _ = srv.accept()
pid, uid, gid = struct.unpack("3i", conn.getsockopt(
    socket.SOL_SOCKET, socket.SO_PEERCRED, struct.calcsize("3i")))
if uid != os.getuid():
    conn.close()                                 # only our own user may talk to the
server
```

The client connects with `socket.AF_UNIX` to the same path. HTTP can run on top unchanged; for example, uvicorn has `--uds`.

Residual risk. No encryption: root, or anything able to attach to either process (ptrace, eBPF), can still read the data. It only works on one machine.

4.3.2 Alternative 2: TLS 1.3 with a self-signed certificate

How it works. The client and server keep using TCP, but wrap it in TLS 1.3 (RFC 8446). The handshake uses ephemeral (EC)DHE key exchange, so every session gets fresh keys and **forward secrecy**. All records are protected with an AEAD cipher (AES-GCM or ChaCha20-Poly1305), which provides confidentiality and integrity together. On loopback, a self-signed certificate is enough, as long as the client **pins** it: it trusts only that exact certificate, not the system CA store.

What it fixes. Reading traffic on `lo` (the capture would show only encrypted TLS records), tampering and forgery (the AEAD tag check fails), server impersonation (certificate pinning), and replay. Identical plaintexts no longer produce identical ciphertexts. HTTP headers and JSON keys are encrypted too.

Setup sketch:

```
openssl req -x509 -newkey ed25519 -nodes -days 365 \
  -keyout server.key -out server.crt \
  -subj "/CN=muhammadrafique" -addext
"subjectAltName=DNS:muhammadrafique,IP:127.0.0.1"
chmod 600 server.key
```

```
import ssl
# server
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.minimum_version = ssl.TLSVersion.TLSv1_3
ctx.load_cert_chain("server.crt", "server.key")
httpd.socket = ctx.wrap_socket(httpd.socket, server_side=True)
# client: trust only this certificate
requests.post("https://muhammadrafique:5000/submit", json=payload,
verify="server.crt")
```

The lab machine's OpenSSL 3.6.3 supports TLS 1.3. For **mutual TLS**, also issue a client certificate and set `ctx.verify_mode = ssl.CERT_REQUIRED` on the server, so only the real client can connect.

Residual risk. The private key must be protected (`0600`, never committed). An observer still sees that a connection exists and roughly how large it is. Certificates need renewal.

4.3.3 Alternative 3 (optional): authenticated encryption in the application

If the transport cannot change, the affine cipher can be replaced inside the JSON with a modern AEAD such as **AES-256-GCM** (NIST SP 800-38D) or **ChaCha20-Poly1305** (RFC 8439), for example `AESGCM` from the Python `cryptography` package. Each message gets a fresh random 96-bit nonce, sent alongside the ciphertext, and decryption fails loudly if a single bit was changed. A nonce must **never** be reused with the same key. The key has to come from a proper exchange or a protected file, not the source code. This fixes confidentiality and integrity of the fields. Unlike TLS, it leaves headers, JSON keys and message timing visible, and makes key management the application's problem.

4.4 Comparison

Option	Threats addressed	Effort	Residual risk
Current affine cipher	None in practice: broken in 0.14 s	—	Everything readable and forgeable
UNIX domain socket, 0600 + `SO_PEERCRED`	Sniffing on `lo`; other local users; spoofed clients	Low: change bind/ connect, stdlib only	No encryption against root or ptrace; single host only
TLS 1.3, self-signed + pinning (opt. mTLS)	Eavesdropping, tampering, impersonation, replay; metadata hidden inside the tunnel	Medium: certificate generation and handling	Key file protection; connection timing and size still visible
AEAD fields (AES-GCM / ChaCha20-Poly1305)	Confidentiality and integrity of the encrypted values	Medium: key management, nonce discipline	Metadata and headers in clear; nonce reuse is catastrophic
UDS + TLS (defence in depth)	All of the above on one host	Medium	Compromise of the user account or root

Recommendation. For a local-only service like PsycoSupport, use a **UNIX domain socket with `0600` permissions and `SO_PEERCRED` checks**: it removes the traffic from the network stack entirely, and it is the smallest change. Where client and server might ever run on separate hosts or VMs, as the upstream README allows, use **TLS 1.3 with a pinned certificate** (and mTLS). In both cases, delete the homemade cipher and its hard-coded key.

4.5 Sources

- E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, August 2018.

- Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, IETF, June 2018.
- M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.
- `unix(7)` (`man 7 unix`), Linux man-pages: `AF_UNIX` sockets, socket file permissions, `SO_PEERCRED`.
- Python Software Foundation, `ssl` module documentation: `SSLContext`, `TLSVersion.TLSv1_3`.
- A. Kerckhoffs, "La cryptographie militaire", *Journal des sciences militaires*, vol. IX, 1883.
- Own measurements: Sections 2–3 of this report (`evidence/task2/`, `report/figures/`).

Ethical Reflection

1. Scope — only our own instance

All traffic capture and testing were performed exclusively on the PsycoSupport instance running on my own machine, in accordance with the lab brief, and were limited to that instance and its loopback traffic. Performing the same activities on systems I do not own or have explicit permission to test would be unethical and potentially illegal. Respecting the defined scope is essential to responsible security testing.

2. Synthetic data only

The five patients, named `tester01` to `tester05`, and all of their mental-health concerns were fictional, created specifically for this lab. No real person's name, mood or private concern was entered, captured or decrypted during the experiment. This mattered especially here because mental-health information is highly sensitive and could cause harm or violate privacy if exposed; synthetic data let me demonstrate the vulnerability without compromising anyone.

3. Responsible credential handling

The upstream application's API key was deliberately ignored and never used, because the entire setup ran locally through Ollama. No secrets were committed to the GitHub repository: the capture files, the `.memory` folder and any `.env` files were excluded with Git ignore rules. Real API keys, passwords and other credentials should never be committed to a public or shared repository, and proper secret management is an essential part of secure development.

4. Responsibility when using security tools

Packet-analysis and traffic-capture tools such as Wireshark, tcpdump and dumpcap are dual-use technologies that can support both legitimate security research and harmful activity. In this lab they were used only on the loopback interface of a machine under my control. Although the techniques showed how a weak encryption scheme could be broken, applying them to another person's traffic without authorization could seriously compromise their privacy and security. Learning these tools therefore demands not only technical skill but also ethical judgement and respect for authorization boundaries.

5. What was really at stake

The experiment demonstrated that the application's weak encryption could be broken in approximately 0.14 seconds, allowing an eavesdropper on the same machine to read patient usernames, moods, private concerns and the advice returned by the application. In a real deployment this would expose highly sensitive health information and undermine users' trust in the service, and such information may receive special-category protection under applicable data-protection laws. If a weakness like this were

discovered in a real system, it should be documented and reported responsibly rather than exploited to access private information.

6. Why we conducted the experiment

The purpose of this lab was to understand how the encryption mechanism failed and to identify appropriate ways to improve security, not to defeat protection for its own sake. The fixes proposed in Task 4 — a UNIX domain socket, or TLS 1.3 where appropriate — show concrete ways to protect the communication, and the right choice depends on the deployment architecture and security requirements. Understanding how security controls fail is what lets developers design, implement and verify stronger protections.

References

1. R. E. Lewand, *Cryptological Mathematics*, Mathematical Association of America, 2000. (English letter frequencies.)
2. Al-Kindi, *Risala fi Istikhraj al-Mu'amma* (A Manuscript on Deciphering Cryptographic Messages), c. 9th century. (Origin of frequency analysis.)
3. E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, August 2018.
4. Y. Nir and A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, IETF, June 2018.
5. M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.
6. A. Kerckhoffs, "La cryptographie militaire", *Journal des sciences militaires*, vol. IX, 1883.
7. `unix(7)`, Linux man-pages project. (`AF_UNIX` sockets, socket-file permissions, `SO_PEERCRED`.)
8. Python Software Foundation, *`ssl` — TLS/SSL wrapper for socket objects*, Python 3 documentation.
9. M. A. Rehman, *PsycoSupport*, GitHub repository, https://github.com/maadilrehman/PsycoSupport (commit `0b1cdbf`).
10. Ollama, https://ollama.com; model `llama3.2` (Meta Llama 3.2, 3B).

Appendix: Reproducing the Results

Everything in this report is produced by scripts in the repository, run through `uv` and wrapped as `task` commands (`go-task`). All runs are offline on Kali Linux over loopback `127.0.0.1`.

A.1 Setup

```
curl -LsSf https://astral.sh/uv/install.sh | sh                        # Python env
manager
sh -c "$(curl -fsSL https://taskfile.dev/install.sh)" -- -b ~/.local/bin   # task
runner
curl -fsSL https://ollama.com/install.sh | sh && ollama pull llama3.2 # local LLM
echo "127.0.0.1 muhammadrafique" | sudo tee -a /etc/hosts             # host file
(own name)
uv sync                                                                # project
dependencies
```

A.2 Pipeline

Step	Command	Output
Run the server	`task server`	HTTP server on `127.0.0.1:5000`
Capture loopback (terminal 1)	`task capture`	`evidence/task2/session_all.pcap`
Send 5 test cases (terminal 2)	`task run-cases`	5 sessions through the real client
Extract ciphertexts	`task extract`	`ciphertexts.csv` (20 rows)
Frequency analysis	`task freq`	`report/figures/` tables + charts
Recover the key	`task recover`	(a, b) = (17, 8), full working
Decrypt everything	`task decrypt`	`report/figures/decrypted.csv`, 5/5 PASS
Build this PDF	`task report`	`report.pdf`

A.3 Scripts

Path	Purpose
`psycare_lab/{server,client,affine}.py`	The app written to the brief (affine cipher + Ollama)
`scripts/capture/test_cases.csv`, `run_cases.py`	Five synthetic sessions and the runner
`scripts/capture/extract_ciphertexts.py`	pcap → `ciphertexts.csv` (standard library only)
`scripts/frequency/{english,count,patterns,plot}.py`	Letter frequencies, patterns, charts
`scripts/cipher/{affine,recover_key,decrypt_all}.py`	Attacker toolkit, key recovery, decryption
`scripts/report/build_report.py`	Assembles this PDF from `report/sections/`

The affine key was recovered from intercepted ciphertext alone (Section 3.2); the key in `psycare_lab/affine.py` was consulted only as a final confirmation.